

Documentação API: Restful-booker

1. Introdução

Este documento tem como objetivo apresentar os testes realizados na API pública [Restful Booker](#) utilizando a ferramenta [Postman](#). A API foi utilizada como ambiente de prática para validação de serviços REST, simulando operações de gerenciamento de reservas de hospedagem.

Durante a execução dos testes, foram validados os principais endpoints da aplicação, contemplando operações de criação, consulta, atualização e exclusão de reservas, além da autenticação necessária para acesso a determinadas funcionalidades. Também foram analisados os códigos de status HTTP, estrutura das respostas e comportamento da API em diferentes cenários.

O objetivo dos testes foi garantir que os serviços disponibilizados pela API apresentassem respostas consistentes, funcionamento adequado e conformidade com o comportamento esperado para cada requisição realizada.

Esta documentação descreve os cenários executados, os endpoints utilizados, os métodos aplicados, os resultados obtidos e as evidências geradas durante os testes realizados no Postman.

2. Objetivo

Validar o ciclo de vida de uma reserva (CRUD) e autenticação de segurança.

3. Escopo dos Testes

Funcionalidades contempladas

Nível 1 — Obrigatório

- Autenticação básica
- CRUD de reservas
- Validação de campos obrigatórios

4. Tipo de Teste

Tipo	Descrição
Teste Funcional	Verificar se cada <i>endpoint</i> da API funciona conforme os requisitos.

5. Ferramentas Utilizadas

Ferramenta	Finalidade
Postman	Teste e automatização de API

6. Ambiente de Testes

Item	Descrição
Ambiente	Homologação
Ferramenta de Teste	Postman
API Testada	Restful Booker
URL Base	https://restful-booker.herokuapp.com
Formato de requisições	JSON
Execução dos testes	Manual via collections
Navegador Principal	Google Chrome
Execução	Local e CI/CD

7. Configurações de ambiente

Para rodar a collection, as seguintes variáveis globais/ambiente devem ser configuradas:

- `base_url`: `https://restful-booker.herokuapp.com`
- `auth_token`: Armazena dinamicamente o token de autenticação gerado pelo `endpoint/auth`.

8. Massa de Dados

Usuários utilizados nos testes

Tipo de usuário	Objetivo
Usuário válido	Validar login
Usuário inválido	Validar login

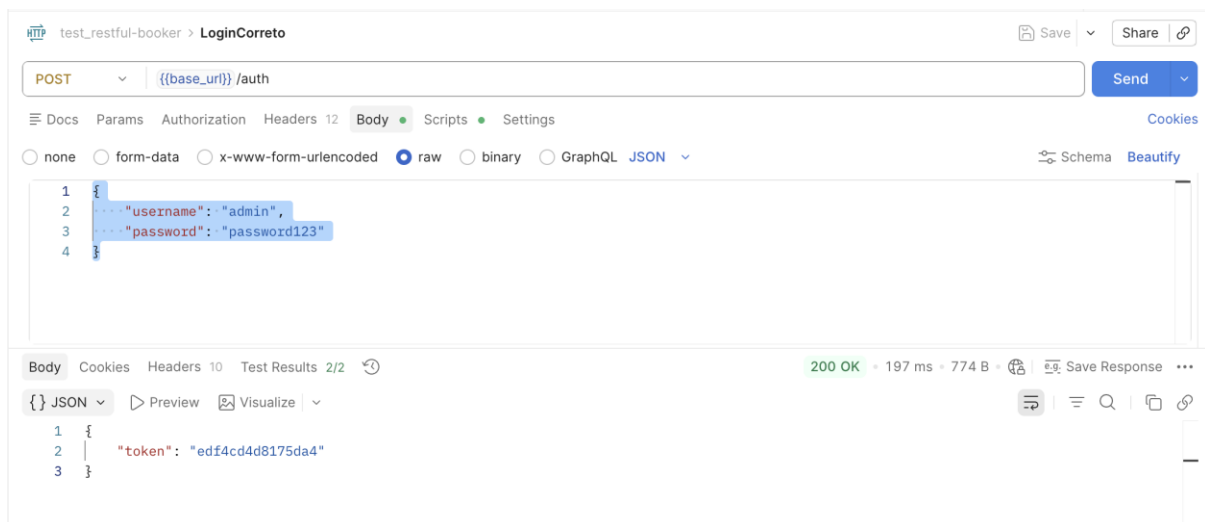
9. Cenários de Teste

Cenário 1: Autenticação (POST /auth)

- **Objetivo:** Gerar token de acesso para endpoints protegidos.
- **Payload (JSON):**

```
{  
  "username": "admin",  
  "password": "password123"  
}
```

Validações Esperadas: Status 200 OK e presença da chave token na resposta.

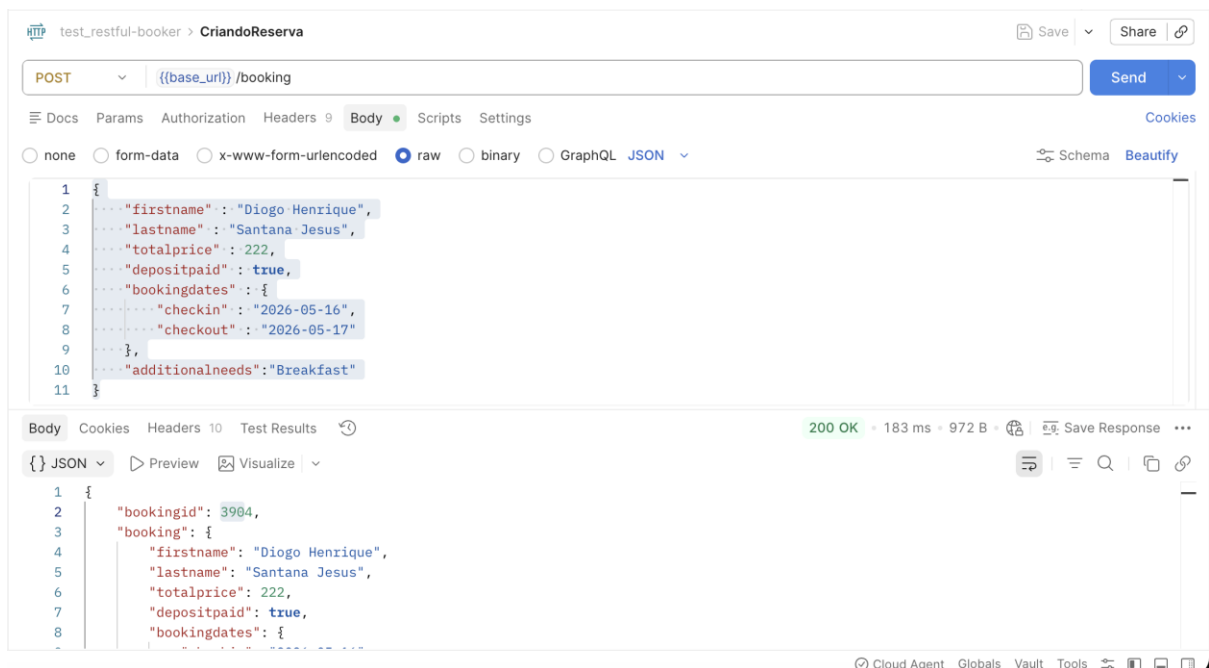


Cenário 2: Criar Reserva (POST /booking)

- **Objetivo:** Criar uma nova reserva com sucesso.
- **Payload (JSON):** Dados estruturados diretamente na raiz (campos: firstname, lastname, totalprice, depositpaid, bookingdates, additionalneeds).

```
{  
  "firstname" : "Diogo Henrique",  
  "lastname" : "Santana Jesus",  
  "totalprice" : 222,  
  "depositpaid" : true,  
  "bookingdates" : {  
    "checkin" : "2026-05-16",  
    "checkout" : "2026-05-17"  
  },  
  "additionalneeds": "Breakfast"  
}
```

- **Validações Esperadas:** Status 200 OK e retorno do ID gerado (bookingid).



Cenário 3: Buscar Reserva por ID (GET /booking/:id)

- **Objetivo:** Retornar os dados exatos da reserva criada no cenário anterior.
- **Validações Esperadas:** Status 200 OK e correspondência dos dados enviados.

The screenshot displays a REST client interface with the following details:

- URL:** `GET {{base_url}}/booking/637`
- Method:** GET
- Body:** 1
- Response:** 200 OK, 176 ms, 935 B. The response body is shown in JSON format:

```
1 {
2   "firstname": "Diogo Henrique",
3   "lastname": "Santana Jesus",
4   "totalprice": 222,
5   "depositpaid": true,
6   "bookingdates": {
7     "checkin": "2026-05-16",
8     "checkout": "2026-05-17"
9   },
10  "additionalneeds": "Breakfast"
11 }
```

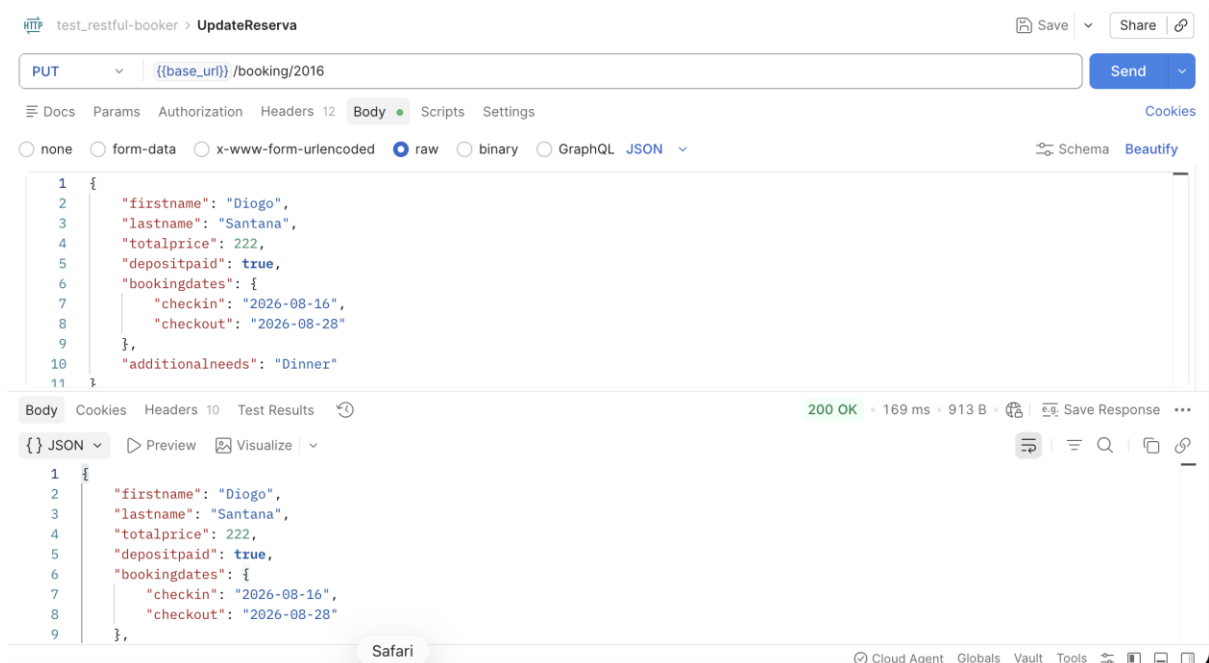
The interface also shows tabs for Body, Cookies, Headers, Test Results, and a Google Chrome browser window at the bottom.

Cenário 4: Atualização Completa (PUT /booking/:id)

- **Objetivo:** Alterar todos os campos de uma reserva ativa.
- **Headers Obrigatórios:**
 - Content-Type: application/json
 - Accept: application/json
 - Cookie: token={{auth_token}} (Nota: Esta API utiliza autenticação baseada em Cookie em vez de Bearer Token tradicional).
- **Payload (JSON):** Corpo plano contendo os novos dados do cliente na raiz do objeto.

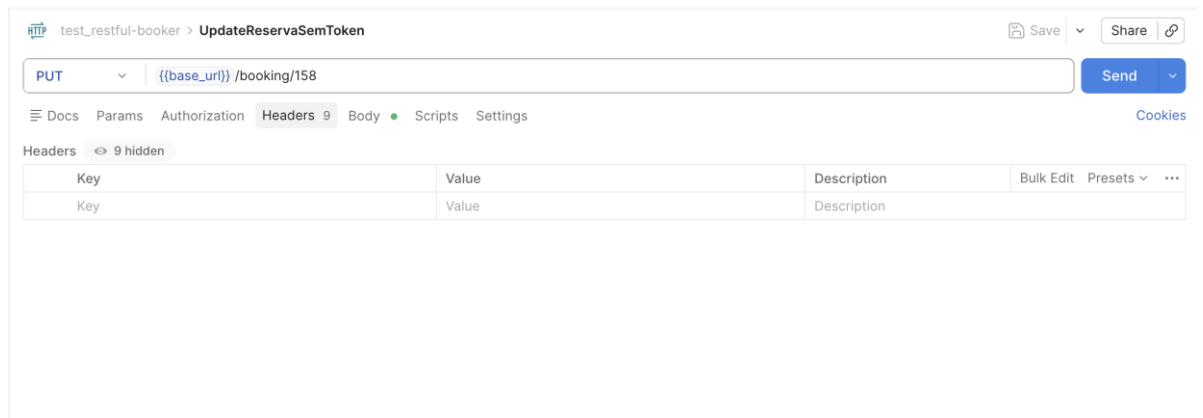
```
{  
  "firstname": "Diogo",  
  "lastname": "Santana",  
  "totalprice": 222,  
  "depositpaid": true,  
  "bookingdates": {  
    "checkin": "2026-08-16",  
    "checkout": "2026-08-28"  
  },  
  "additionalneeds": "Dinner"  
}
```

- **Validações Esperadas:** Status 200 OK e dados atualizados no corpo da resposta.

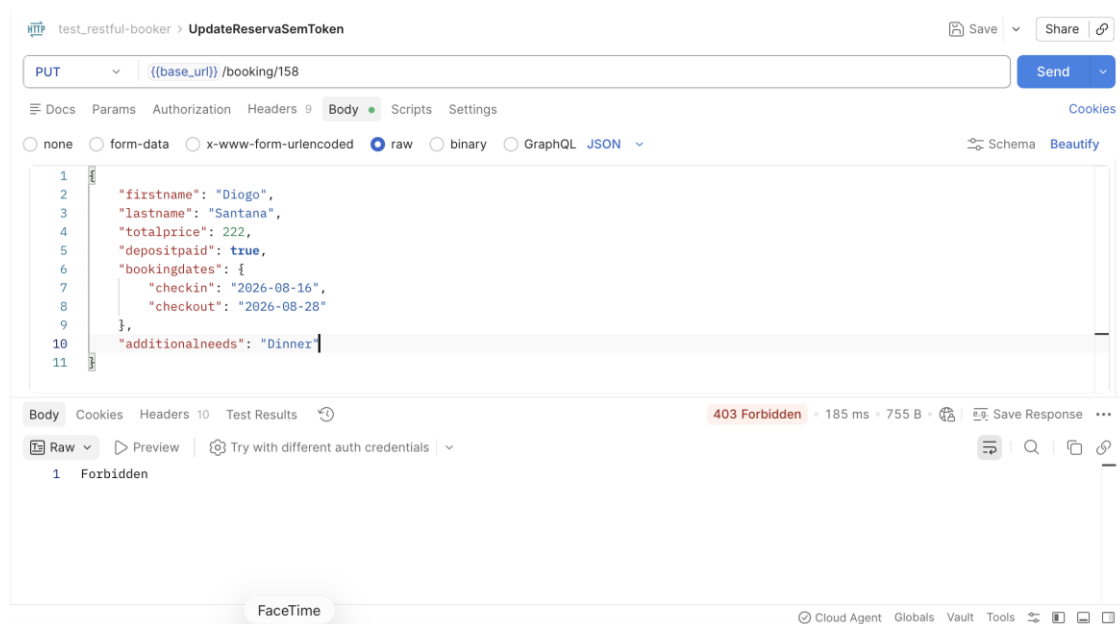


Cenário 5: Teste de Exceção / Segurança (PUT sem Token)

- **Objetivo:** Garantir a restrição de acesso a chamadas não autorizadas.
- **Condição:** Executar o PUT omitindo ou enviando um cabeçalho Cookie inválido.



Validações Esperadas: Status 403 Forbidden ou 400 Bad Request (caso o payload esteja corrompido).



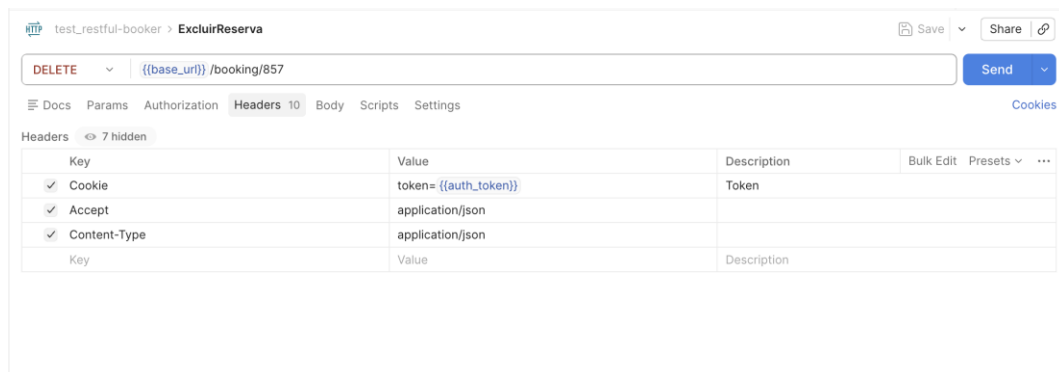
Cenário 6: Exclusão de Reserva (DELETE /booking/:id)

Objetivo: Remover uma reserva existente do banco de dados de forma definitiva.

Headers Obrigatórios:

Content-Type: application/json

Cookie: token={{auth_token}} *(Exige privilégios de administrador)*



Parâmetros de URL: ID da reserva a ser excluída (ex: /booking/2669).

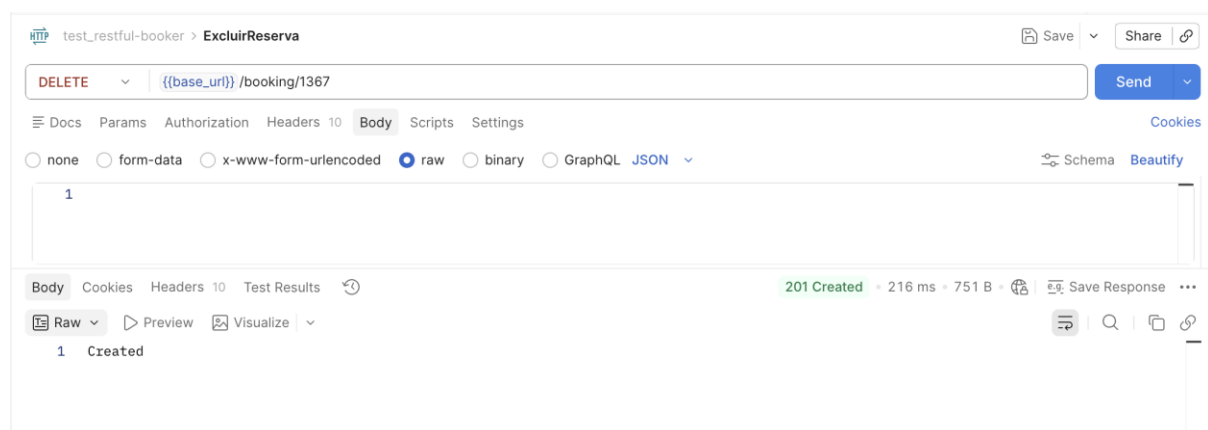
Validações Esperadas (Cenário Positivo):

Status **201 Created** (Comportamento específico desta API Restful-Booker para deleções bem-sucedidas).

Mensagem de texto no corpo da resposta: Created.

Fluxo de Pós-Condição:

Executar uma chamada GET /booking/:id com o mesmo ID para garantir o retorno do status **404 Not Found**, confirmando a exclusão física do registro.



10. Critérios de Aceite

Funcionalidade	Critério
Login válido	Usuário autenticado
Login inválido	Mensagem de erro exibida
Validação de reservas	Busca de todos os ID
Validação reserva por ID	Busca de reserva por ID
Atualização de dados da Reserva	Corrigir informações incorretas
Exclusão de reserva	Exclusão de reserva com informação incorreta

11. Análise de bugs

Após algumas validações notei que as informações da base depois de um curto tempo entre 4 e 5 minutos, as reservas que foram enviadas, elas somem. Isso faz com que tenha que fazer diversos envios de reservas para eu seguir com algo que eu criei em poucos minutos atrás.

13. Priorização dos Testes

Prioridade	Funcionalidade
Alta	Login
Média	Validação da reserva
Média	Busca de reserva
Alta	Criação da reserva
Alta	Atualização da reserva
Alta	Exclusão da reserva

14. Riscos Identificados

Risco	Impacto
Massa de dados inconsistente	Instabilidade

15. Melhorias Futuras

- Aumento do tempo de validação do token

17. Cronograma

Mudança	Data de criação	Versão	Responsável
Criação do plano de testes	12/05/2026	0.0.1	Diogo H. Santana de Jesus
Correção do plano de testes	14/05/2026	0.0.2	Diogo H. Santana de Jesus
Revisão	14/05/2026	0.0.2	Diogo H. Santana de Jesus

17. Responsáveis

Papel / Função	Atribuição Principal no Plano de Testes	Responsabilidade (RACI)	Nome do Profissional
Autor das Instruções	Fornecimento do roteiro de testes, critérios de aceitação e regras de negócio exigidas.	C / A	BeTalent
Analista de teste / QA	Interpretação das instruções recebidas, execução dos cenários práticos e registro de evidências.	R	Diogo H. Santana de Jesus
Homologador	Validação do relatório final de testes para autorização do deploy/entrega do projeto.	A	BeTalent

18. Conclusão

A execução do plano de testes práticos na API **Restful-Booker** foi concluída com sucesso, cobrindo com êxito todo o ciclo de vida dos dados de uma reserva (**CRUD**) e validando as camadas críticas de segurança da aplicação.

Os principais resultados e competências consolidadas neste projeto incluem:

- **Validação de Negócio e Segurança:** Todos os cenários felizes (criação, leitura, atualização e deleção) foram executados com sucesso após o mapeamento correto do modelo de autenticação proprietário da API (via Cookie: token=).
- **Resiliência e Análise de Erros:** Foram identificadas e tratadas de forma proativa as respostas de erro do servidor, como o tratamento do status 403 Forbidden (ajuste na estrutura de autenticação) e 400 Bad Request (correção da raiz estrutural do payload JSON), demonstrando capacidade analítica no mapeamento de requisições.
- **Qualidade e Conformidade:** O fluxo foi encerrado garantindo a integridade dos dados, validando inclusive a pós-condição do cenário de exclusão (DELETE), o que comprova que a aplicação não mantém registros órfãos ou inconsistentes no banco de dados.

Com este ecossistema de requisições mapeado no Postman, a suíte de testes encontra-se pronta e estruturada de forma escalável para a inclusão de scripts de asserção automatizada e testes de regressão contínuos.